

## 2.1 ROS 2 节点 (Node) 基础

**所属阶段：** 第二阶段 · ROS 2 核心通信机制

**讲次数量：** 10 节

**预计时长：** ~45 分钟

**前置依赖：** 1.3 开发环境搭建

**约定 (全文统一)：**

- 工作空间： `~/ros2_ws` (1.3 已创建)；构建在根目录执行，运行前 `source install/setup.bash`
- 创建 Package： **必须** `cd ~/ros2_ws/src` 后 `ros2 pkg create ...` (禁止手写目录拼凑)
- 运行节点： `ros2 run` 包名 可执行名 (不用 `loose` 的 `python3 ~/xxx.py`)
- `colcon`、`overlay`、排错：见 **1.3**，本节不重复

### 学习目标

完成本节后，学员将能够：

- 描述节点概念，用 `ros2 node list/info` 观察拓扑
- 用 `rcldpy` / `rcldcpp` 编写 OOP 节点 (了解可选函数式写法)
- 在 `~/ros2_ws` 内通过 `ros2 pkg create` 创建 Python / C++ Package 并 `ros2 run` 启动节点
- 运用 OOP 初始化分区模板与社区最佳实践

## 本节路径

---

| 阶段 | 讲次   | 内容                                                                                    |
|----|------|---------------------------------------------------------------------------------------|
| 概念 | 1    | 节点、命名空间、Package 关系                                                                    |
| 代码 | 2-5  | Python / C++ 节点写法 (理解 API, 讲次 7-8 打包运行)                                               |
| 规范 | 6    | OOP 模板与最佳实践                                                                           |
| 工程 | 7-8  | <code>ros2 pkg create</code> → 配置 → <code>colcon build</code> → <code>ros2 run</code> |
| 参考 | 9-10 | colcon 要点、客户端库选型                                                                      |

---

# 讲次 1：什么是 ROS 2 节点

## 知识点

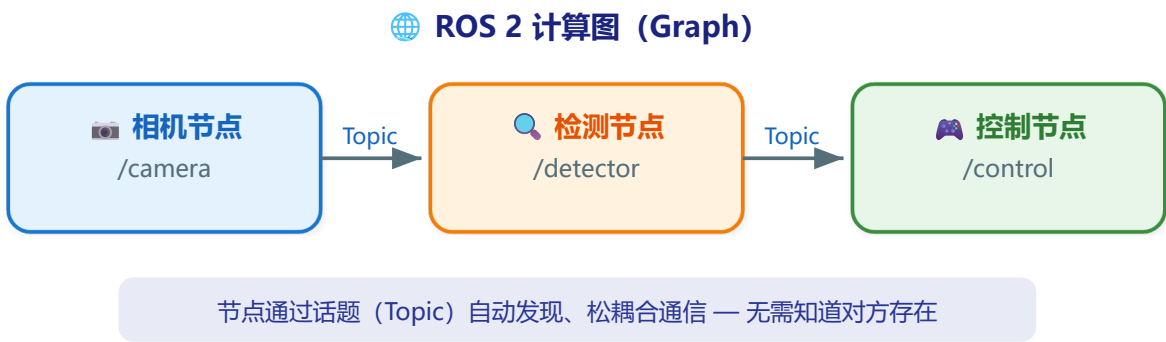
在 ROS 2 中，**节点 (Node)** 是参与 ROS 图并执行计算的逻辑单元，负责一项明确任务，例如读取传感器、控制电机、规划路径或可视化数据。节点不等同于操作系统进程：初学示例通常一个进程运行一个节点，组件化系统也可以在同一进程中组合多个节点。

可以把 ROS 2 系统想象成一支团队：每个节点是一个专职员工，大家通过统一的通信机制（话题、服务、动作）协作，而不是把所有逻辑塞进一个巨大的程序里。

## 节点的核心特征

| 特征   | 说明                                                |
|------|---------------------------------------------------|
| 部署方式 | 可一进程一节点，也可把多个组件节点组合到同一进程；隔离性取决于部署方式               |
| 名称   | 节点全名应保持唯一（可用命名空间区分）；ROS 2 不会禁止重名，但 CLI、参数和日志会产生歧义 |
| 通信端点 | 节点通过 Publisher、Subscriber、Service、Action 与外界交互    |
| 可分布式 | 节点可以运行在同一台机器，也可以分布在局域网内多台机器上                      |

## 节点在系统中的位置



## 用命令行观察节点

你在 1.3 中已通过 Turtlesim 启动了第一个节点。此处聚焦**节点专属** CLI，用于观察计算图拓扑：

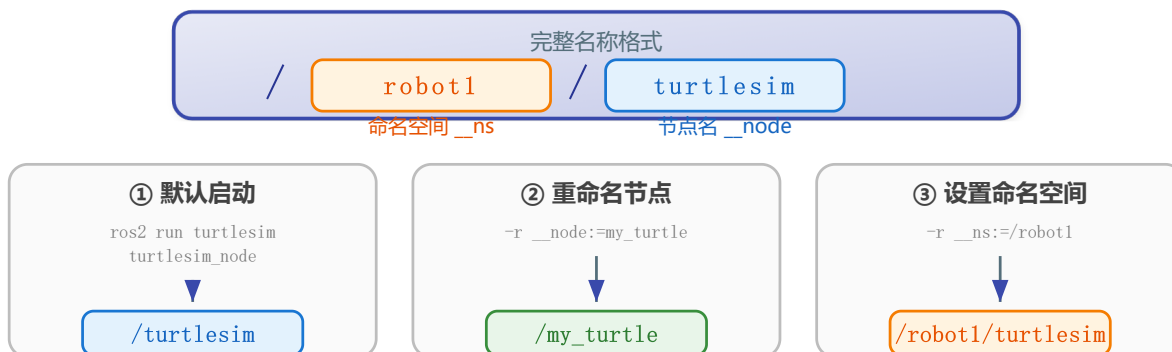
```
ros2 node list          # 列出当前活跃节点
ros2 node info /turtlesim # 查看指定节点的发布/订阅/服务
```

`node info` 输出中的话题、服务列表，构成了 ROS 2 的**计算图 (Computation Graph)**。后续讲次 2-8 编写自己的节点后，同样用这两条命令验证节点是否注册成功。

## 节点名称与命名空间

节点名称是在创建时指定的字符串标识符，可在**运行时**通过 `--ros-args` 重映射，无需修改源码。命名空间 (Namespace) 用于组织层级，多机器人场景中可避免同名节点冲突。

### 📌 节点全名构成与运行时重映射



多机器人：同名节点，不同命名空间 → 互不冲突



默认命名空间为 /，全名可写为 /节点名；工程中应避免重名造成 CLI、参数和日志歧义

```
# 运行时重命名节点（不影响代码中的默认值）
ros2 run turtlesim turtlesim_node --ros-args -r __node:=my_turtle
ros2 node list
# 输出：/my_turtle
```

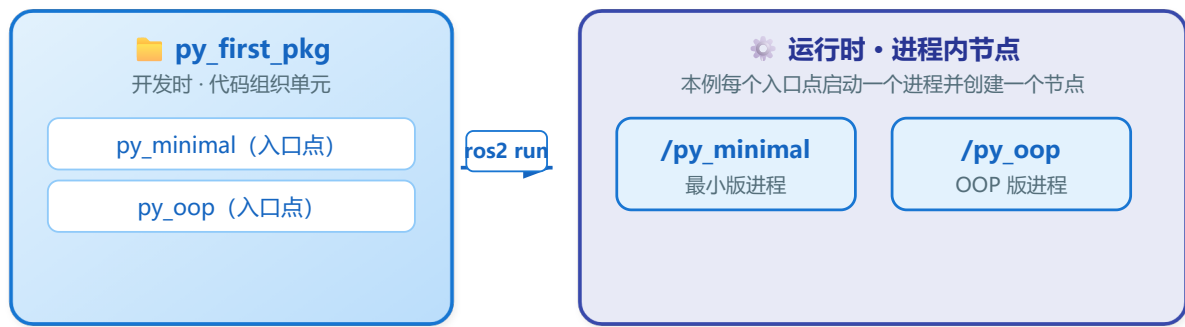
```
# 设置命名空间，组织节点层级
ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/robot1
ros2 node list
# 输出：/robot1/turtlesim
```

## 节点与 Package 的关系

节点是**运行时逻辑概念**，Package 是**开发时组织单元**。一个 Package 可以提供多个可执行文件或 Python 入口点；一个入口点通常在新进程中创建一个节点，也可以创建多个节点。组件容器还能把多个可组合节点加载到同一进程。初学示例使用 `ros2 run` 启动单节点进程：

```
ros2 run py_first_pkg py_minimal # 包名 + 可执行名 → 启动进程并创建节点
ros2 run py_first_pkg py_oop
```

## 📦 节点与 Package 的关系



💡 一个 Package 可含多个节点; `ros2 run pkg_name node_name` 将代码变为运行中的进程

### 为什么节点是第一课

后续 Topic、Service、Action、Parameter 均依附于节点; 掌握节点创建与 `ros2 run` 启动, 是进入 ROS 2 编程的入口。

---

## 讲次 2：【PY】函数式节点写法（可选）

### 知识点

1.3 已通过 `hello_ros2` 完成 OOP 首节点。本节补充一种**不继承 class** 的函数式写法——在 `main()` 中直接 `Node("name")`，适合快速理解 API；**正式开发与运行均走 Package**（讲次 7）。

### 代码结构

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

def main(args=None):
    rclpy.init(args=args)
    node = Node("my_first_node")
    node.get_logger().info("Hello ROS 2!")
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

完整可运行版本写入 `~/ros2_ws/src/py_first_pkg/` 后，用 `ros2 run py_first_pkg py_minimal` 启动（讲次 7）。仓库参考：`stage2_code/py_first_pkg/`。

### 函数式 vs OOP

| 对比项 | 函数式（本讲）                                           | OOP（1.3 / 讲次 3）                    |
|-----|---------------------------------------------------|------------------------------------|
| 结构  | <code>main()</code> 中直接 <code>Node("name")</code> | 继承 <code>Node</code> 类             |
| 适用  | 理解 API、一次性脚本                                      | 正式项目、多回调节点                         |
| 运行  | <code>ros2 run</code> （Package 入口）                | <code>ros2 run</code> （Package 入口） |

实际开发推荐 OOP。

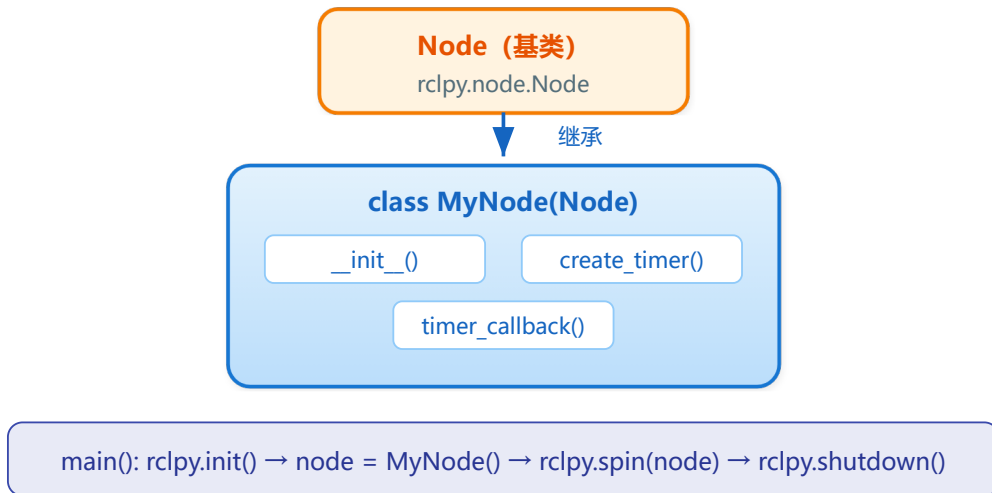
## 讲次 3：【PY】 OOP 节点深化

### 知识点

1.3 的 TalkerNode 已演示 OOP + 定时器 + Publisher 的完整流程。本节**暂不涉及 Publisher**（话题通信见 2.2），专注节点自身结构、定时器回调与命名规范——这也是后续所有实验的标准模板。

### 设计思路

#### OOP 版节点设计思路



继承 `Node` 后，`self` 本身就是节点实例，可以直接调用 `self.create_publisher()`、`self.create_timer()` 等 API。

## 完整代码

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__("py_test")
        self.counter_ = 0
        self.get_logger().info("Hello world")
        self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info("Hello " + str(self.counter_))
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = MyNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

## 代码说明

与 1.3 TalkerNode 的差异：去掉 Publisher，仅保留定时器；成员变量推荐尾缀 `_`（如 `self.counter_`）。  
`create_timer(period_sec, callback)` 在 `__init__` 中初始化（模板见讲次 6）。

可运行版本：`~/ros2_ws/src/py_first_pkg/py_first_pkg/oop_node.py` → `ros2 run py_first_pkg py_oop`（讲次 7）。

## 讲次 4：【C++】编写第一个节点（最小版本）

### 知识点

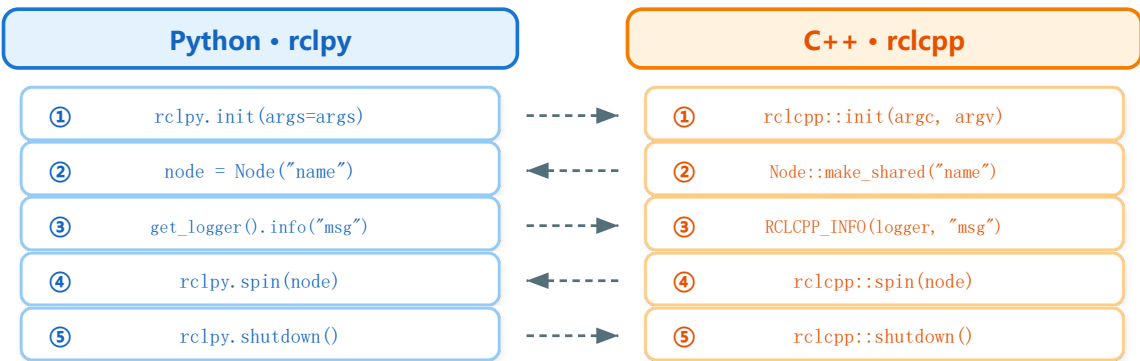
C++ 是 ROS 2 的核心开发语言，rclcpp 提供了与 rclpy 高度对称的 API。本讲编写 C++ 最小版节点——在 `main()` 中直接创建节点，无需定义 class。

### 核心 API 概览

| 函数 / 类                                | 作用                                  |
|---------------------------------------|-------------------------------------|
| <code>rclcpp::init(argc, argv)</code> | 初始化 rclcpp                          |
| <code>rclcpp::Node("name")</code>     | 创建节点（通常用 <code>make_shared</code> ） |
| <code>RCLCPP_INFO(logger, ...)</code> | 输出 INFO 级别日志                        |
| <code>rclcpp::spin(node)</code>       | 进入事件循环                              |
| <code>rclcpp::shutdown()</code>       | 清理资源                                |

### 五步对称流程

#### Python 与 C++ 最小版 · 五步对称流程



⚠ C++ 不能 python3 直接运行，必须 Package + colcon 构建（见讲次 8）

### 完整代码

创建文件 `minimal_node.cpp`：

```
#include "rclcpp/rclcpp.hpp"

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("my_first_node");
    RCLCPP_INFO(node->get_logger(), "Hello ROS 2!");
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

## 代码说明

与 Python 最小版对称: `init` → `Node::make_shared` → `RCLCPP_INFO` → `spin` → `shutdown`。C++ 节点用 `shared_ptr` 管理生命周期; `RCLCPP_*` 是宏, 支持编译期裁剪日志级别。

## 编译与运行

C++ 节点必须通过 `ros2 pkg create + colcon` 构建后 `ros2 run` (见讲次 8)。本节先理解代码结构与 API 对称关系。

## 预期输出

```
[INFO] [my_first_node]: Hello ROS 2!
```

(五步 API 对照见上方 SVG; 不再重复表格。)

---

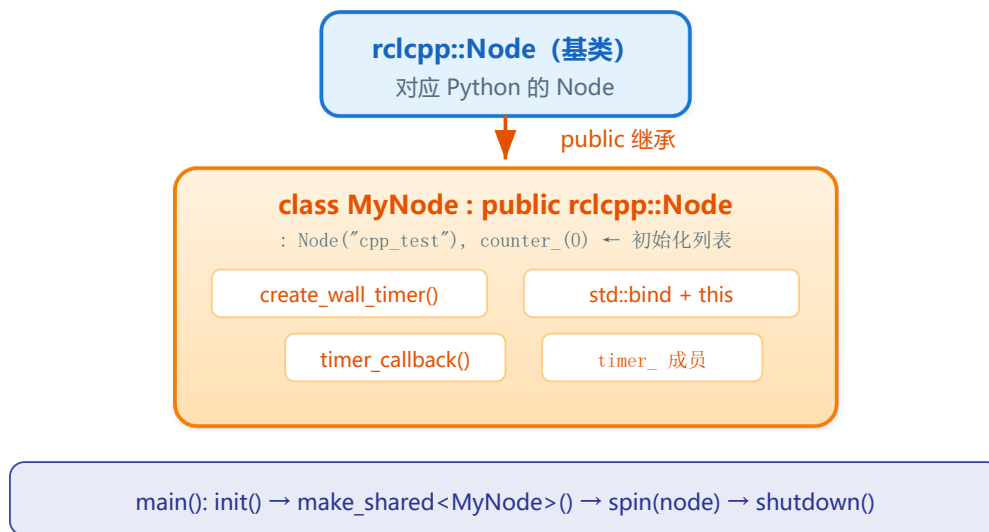
## 讲次 5: 【C++】编写第一个节点 (OOP 版本)

### 知识点

C++ OOP 版节点继承 `rclcpp::Node`，结构与 Python OOP 版完全对应。这是 C++ ROS 2 开发的标准写法。

### 设计思路

#### C++ OOP 版节点设计思路



## 完整代码

```
#include "rclcpp/rclcpp.hpp"

class MyNode : public rclcpp::Node
{
public:
    MyNode() : Node("cpp_test"), counter_(0)
    {
        RCLCPP_INFO(this->get_logger(), "Hello world");
        timer_ = this->create_wall_timer(
            std::chrono::seconds(1),
            std::bind(&MyNode::timer_callback, this));
    }

private:
    void timer_callback()
    {
        RCLCPP_INFO(this->get_logger(), "Hello %d", counter_++);
    }

    rclcpp::TimerBase::SharedPtr timer_;
    int counter_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<MyNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

## 代码说明

继承 `rclcpp::Node`，用初始化列表 `: Node("cpp_test")` 注册节点名（对应 Python 的 `super().__init__()`，必须最先完成）。

## 两种定时器对比

| 类型         | 创建方法                             | 时间源         | 适用场景           |
|------------|----------------------------------|-------------|----------------|
| Wall Timer | <code>create_wall_timer()</code> | 系统墙钟（真实时间）  | 绝大多数场景         |
| ROS Timer  | <code>create_timer()</code>      | ROS 时钟（可仿真） | Gazebo 仿真、仿真时间 |

本课程默认使用 `create_wall_timer()`。涉及 Gazebo 仿真时再切换到 `create_timer()`（ROS 时钟）。构建与运行步骤见讲次 8。

## 四个常见陷阱

| # | 陷阱             | 正确做法                                                                                  |
|---|----------------|---------------------------------------------------------------------------------------|
| ① | 成员函数作定时器回调     | 必须用 <code>std::bind(&amp;MyNode::timer_callback, this)</code> 绑定 <code>this</code> 指针 |
| ② | 定时器句柄未保存       | <code>timer_</code> 必须声明为成员变量，否则构造函数结束后立即析构                                           |
| ③ | 基类 Node 未正确初始化 | 初始化列表中 <code>: Node("cpp_test")</code> 必须放在第一位                                        |
| ④ | 节点实例创建方式错误     | 用 <code>std::make_shared&lt;MyNode&gt;()</code> 创建，再传入 <code>spin()</code>            |

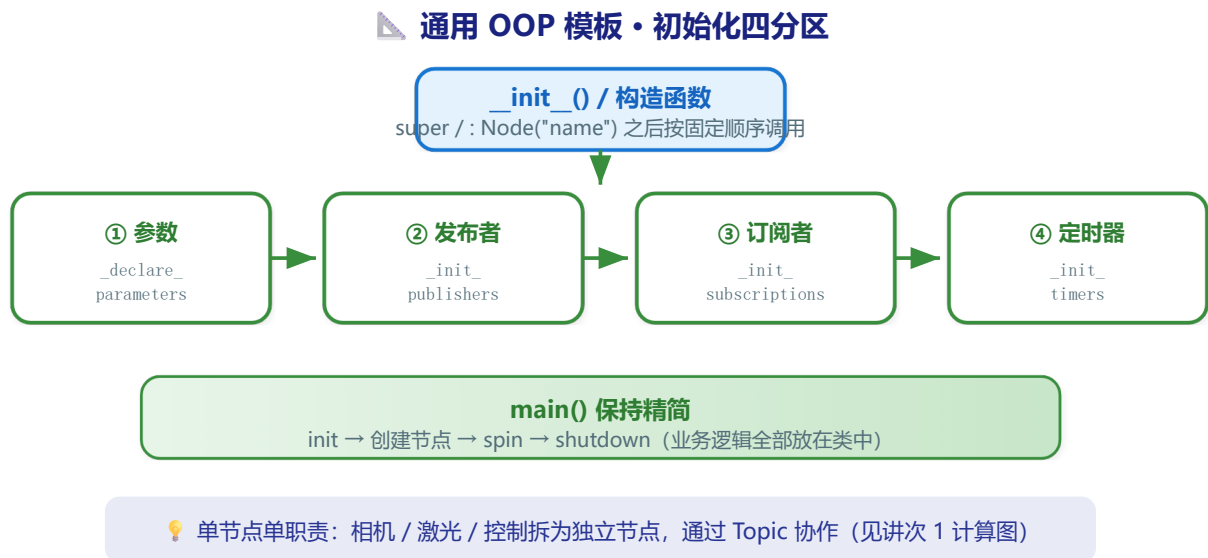
---

## 讲次 6：节点 OOP 模板与最佳实践

### 知识点

本讲提炼一个**通用 OOP 节点模板**，涵盖 Python 和 C++ 双语言版本，并总结 ROS 2 社区公认的最佳实践。后续课程的所有节点实验都基于此模板。

### 初始化四分区结构



### 模板骨架

讲次 7 / 8 的 Package 中采用此结构；完整文件见 `~/ros2_ws/src/py_first_pkg/` 与 `cpp_first_pkg/`。

**Python:** `super()` → `_declare_parameters` → `_init_publishers` → `_init_subscriptions` → `_init_timers` → `main()` 仅 `init` / `spin` / `shutdown`。

**C++:** 初始化列表 `: Node("name")` → 同上四步 → `main()` 仅 `init` / `make_shared` / `spin` / `shutdown`。

最佳实践清单

| 实践      | 要点                                                                                                                                                 |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 节点命名    | 小写 + 下划线、描述功能 (如 <code>lidar_processor</code> ) , 避免 <code>node1</code> 或驼峰                                                                        |
| 初始化分区   | <code>__init__</code> 拆为 <code>_declare_parameters</code> / <code>_init_publishers</code> / ... (见上方模板)                                            |
| 成员变量    | Python 尾缀 <code>_</code> ; C++ 用 <code>SharedPtr</code> 成员保存 timer / publisher                                                                     |
| 资源生命周期  | Python 在 <code>finally</code> 中 <code>destroy_node()</code> ; C++ 依赖智能指针 RAII                                                                      |
| 日志      | <code>get_logger().info()</code> / <code>RCLCPP_INFO</code> , 按 <code>debug</code> / <code>info</code> / <code>warn</code> / <code>error</code> 分级 |
| 单节点单职责  | 相机、激光、控制拆为独立节点, 通过 Topic 协作 (见讲次 1 计算图)                                                                                                            |
| main 函数 | 只负责 <code>init</code> → 创建节点 → <code>spin</code> → <code>shutdown</code> , 业务逻辑放在类中                                                                |

Package 目录与入口配置见讲次 7 (Python) 、讲次 8 (C++) 。

---

# 讲次 7：【PY】创建 Package

## 知识点

在 `~/ros2_ws` 内用 `ros2 pkg create` 创建 Python Package ( `ament_python` ) , 配置入口点后 `colcon build` + `ros2 run` 。

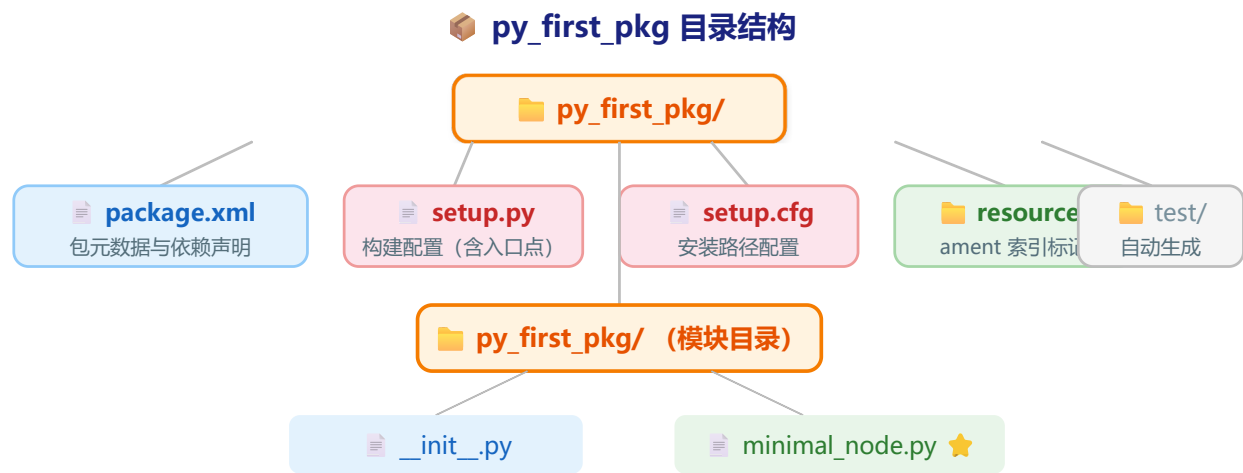
### 1. 创建 Package

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python py_first_pkg --dependencies rclpy
```

| 参数                          | 值                         | 说明       |
|-----------------------------|---------------------------|----------|
| <code>--build-type</code>   | <code>ament_python</code> | Python 包 |
| 包名                          | <code>py_first_pkg</code> | 小写 + 下划线 |
| <code>--dependencies</code> | <code>rclpy</code>        | 运行时依赖    |

仓库完整参考： `stage2_code/py_first_pkg/` → 复制到 `~/ros2_ws/src/` 后 build。

### 2. 目录结构



### 3. 编写节点 (讲次 2–3 代码落地)

将讲次 2–3 的代码写入 `~/ros2_ws/src/py_first_pkg/py_first_pkg/` (节点名 `py_minimal` / `py_oop`) :

`minimal_node.py` (最小版) :

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

def main(args=None):
    rclpy.init(args=args)
    node = Node("py_minimal")
    node.get_logger().info("Hello from py_first_pkg!")
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

oop\_node.py (OOP 版) :

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class OopNode(Node):
    def __init__(self):
        super().__init__("py_oop")
        self.counter_ = 0
        self.get_logger().info("OOP node started")
        self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info("Hello " + str(self.counter_))
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = OopNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

#### 4. 配置 setup.py (入口点)

entry\_points.console\_scripts 将模块 main 注册为 ros2 run 可执行名:

```
entry_points={
    'console_scripts': [
        'py_minimal = py_first_pkg.minimal_node:main',
        'py_oop = py_first_pkg.oop_node:main',
    ],
},
```

格式: '可执行名 = 模块路径:函数名'。每增一个节点, 添加一行。

## 5. 构建与运行

```
cd ~/ros2_ws
colcon build --symlink-install --packages-select py_first_pkg
source install/setup.bash

ros2 run py_first_pkg py_minimal
ros2 run py_first_pkg py_oop
ros2 node list    # /py_minimal 或 /py_oop
```

package.xml 中确认 `<depend>rclpy</depend>` ; colcon 排错见 1.3。修改 entry\_points 后需重新 build。

## 讲次 8: 【C++】创建 Package

### 知识点

在 `~/ros2_ws` 内用 `ros2 pkg create --build-type ament_cmake` 创建 C++ Package, 配置 `CMakeLists.txt` 后 `colcon build` + `ros2 run`。

### 1. 创建 Package

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake cpp_first_pkg --dependencies rclcpp
```

仓库完整参考: `stage2_code/cpp_first_pkg/` → 复制到 `~/ros2_ws/src/` 后 build。

### 2. 目录结构

创建后的目录结构:



### 3. 编写节点 (讲次 4-5 代码落地)

路径均相对于 `~/ros2_ws/src/cpp_first_pkg/`。将讲次 4-5 代码写入 `src/` (节点名 `cpp_minimal` / `cpp_oop`) :

```
#include "rclcpp/rclcpp.hpp"

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("cpp_minimal");
    RCLCPP_INFO(node->get_logger(), "Hello from cpp_first_pkg!");
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

创建 `cpp_first_pkg/src/oop_node.cpp` (OOP 版) :

```
#include "rclcpp/rclcpp.hpp"

class OopNode : public rclcpp::Node
{
public:
    OopNode() : Node("cpp_oop"), counter_(0)
    {
        RCLCPP_INFO(this->get_logger(), "OOP node started");
        timer_ = this->create_wall_timer(
            std::chrono::seconds(1),
            std::bind(&OopNode::timer_callback, this));
    }

private:
    void timer_callback()
    {
        RCLCPP_INFO(this->get_logger(), "Hello %d", counter_++);
    }

    rclcpp::TimerBase::SharedPtr timer_;
    int counter_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<OopNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

#### 4. 配置 CMakeLists.txt

每个节点三行核心配置 ( `add_executable` 名 = `ros2 run` 可执行名) :

```
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)

add_executable(cpp_minimal src/minimal_node.cpp)
ament_target_dependencies(cpp_minimal rclcpp)
install(TARGETS cpp_minimal DESTINATION lib/${PROJECT_NAME})

add_executable(cpp_oop src/oop_node.cpp)
ament_target_dependencies(cpp_oop rclcpp)
install(TARGETS cpp_oop DESTINATION lib/${PROJECT_NAME})

ament_package()
```

C++ 包 `package.xml` 需 `<buildtool_depend>ament_cmake</buildtool_depend>` 与 `<depend>rclcpp</depend>`（`ros2 pkg create` 已自动生成框架）。

5. 构建与运行

```
cd ~/ros2_ws
colcon build --packages-select cpp_first_pkg
source install/setup.bash

ros2 run cpp_first_pkg cpp_minimal
ros2 run cpp_first_pkg cpp_oop
ros2 node list
```

Python vs C++ Package 对比

| 配置项    | Python (ament_python)                            | C++ (ament_cmake)                                    |
|--------|--------------------------------------------------|------------------------------------------------------|
| 创建命令   | <code>--build-type ament_python</code>           | <code>--build-type ament_cmake</code>                |
| 核心配置文件 | <code>setup.py</code>                            | <code>CMakeLists.txt</code>                          |
| 入口声明   | <code>entry_points.console_scripts</code>        | <code>add_executable</code> + <code>install</code>   |
| 依赖声明   | <code>package.xml</code> + <code>setup.py</code> | <code>package.xml</code> + <code>find_package</code> |
| 源码位置   | <code>py_first_pkg/*.py</code>                   | <code>src/*.cpp</code>                               |
| 修改后    | symlink 模式下 Python 代码 <b>无需</b> build            | 必须 <code>colcon build</code> 重新编译                    |

## 讲次 9: colcon 构建要点

| 场景           | 命令 (在 <code>~/ros2_ws</code> 执行)                                  |
|--------------|-------------------------------------------------------------------|
| 日常只改一个包      | <code>colcon build --symlink-install --packages-select PKG</code> |
| 修改 Python 源码 | symlink 模式下重启节点即可 (改 <code>entry_points</code> 除外)                |
| 修改 C++ 源码    | <b>必须重新</b> <code>colcon build --packages-select PKG</code>       |

完整 colcon 流程与排错见 1.3。

## 讲次 10：客户端库选型（概要）

---

| 库      | 语言     | 场景              |
|--------|--------|-----------------|
| rclpy  | Python | 教学、原型、AI 集成     |
| rclcpp | C++    | 生产、Nav2、MoveIt  |
| rcl    | C      | MCU / micro-ROS |

不同语言节点经 DDS **直接互通**（消息类型一致即可）。本课程主路径：**rclpy + rclcpp**。架构分层见 **1.2**。

---

## 本节总结

**工程主线：** `cd ~/ros2_ws/src` → `ros2 pkg create` → 编写节点 → `colcon build` → `source`  
`install/setup.bash` → `ros2 run`

- **节点概念：** 最小计算单元； `ros2 node list/info` 观察拓扑
- **代码写法：** Python / C++ OOP 为主；讲次 2-5 学 API，讲次 7-8 在 Package 中运行
- **Package 入口：** Python → `setup.py` `entry_points`；C++ → `CMakeLists.txt` `add_executable` + `install`

## 命令速查

| 命令                                                                              | 说明          |
|---------------------------------------------------------------------------------|-------------|
| <code>ros2 pkg create --build-type ament_python PKG --dependencies rclpy</code> | 创建 Python 包 |
| <code>ros2 pkg create --build-type ament_cmake PKG --dependencies rclcpp</code> | 创建 C++ 包    |
| <code>colcon build --symlink-install --packages-select PKG</code>               | 构建指定包       |
| <code>ros2 run PKG EXEC</code>                                                  | 运行节点        |
| <code>ros2 node list</code> / <code>ros2 node info /name</code>                 | 查看节点拓扑      |

## 节点 API 速查

| 步骤   | Python (rclpy)                          | C++ (rclcpp)                                        |
|------|-----------------------------------------|-----------------------------------------------------|
| 初始化  | <code>rclpy.init(args=args)</code>      | <code>rclcpp::init(argc, argv)</code>               |
| 创建节点 | <code>class MyNode(Node)</code>         | <code>class MyNode : public rclcpp::Node</code>     |
| 定时器  | <code>self.create_timer(sec, cb)</code> | <code>create_wall_timer(..., std::bind(...))</code> |
| 事件循环 | <code>rclpy.spin(node)</code>           | <code>rclcpp::spin(node)</code>                     |
| 清理   | <code>rclpy.shutdown()</code>           | <code>rclcpp::shutdown()</code>                     |

## 下一步

下一节 **2.2 话题 (Topic) 与发布-订阅**：为节点添加 Publisher / Subscriber，编写第一个多节点应用。